

Mini Project 2. Eksperimen Sampling dan Kuantisasi Audio

Referensi

Mini Project 2 melanjutkan langsung Mini Project 1. Mahasiswa tidak membuat folder baru dan tidak merekam ulang audio. Eksperimen menggunakan `config/project_config.json` dan `data/raw/nyala_001.wav` yang sudah dibuat pada Bab 1. Tujuannya adalah melihat secara terukur bagaimana penurunan sampling rate dan bit depth mengurangi ukuran data sekaligus menambah distorsi. Keputusan format audio dari mini project ini akan menjadi konfigurasi baku untuk analisis FFT/STFT pada Mini Project 3.

Tujuan dan Handoff dari Mini Project 1

Sebelum mulai, pastikan folder `audio_ai_embedded_project` yang sama masih digunakan. Mini Project 2 hanya boleh dijalankan jika `project_status.json` menyatakan status `ready_for_mp02`. Audio referensi tetap kata “nyala” berdurasi sekitar dua detik pada 16 kHz, mono, PCM 16-bit. Dalam 20–25 menit mahasiswa akan membuat tiga variasi format, menghitung metrik sederhana, memilih format kerja proyek, dan memperbarui status untuk Mini Project 3.

Hasil Wajib Mini Project 2

- `data/processed/mp02_nyala_8k16.wav`, yaitu versi 8 kHz/16-bit.
- `data/processed/mp02_nyala_16k8.wav`, yaitu versi 16 kHz/8-bit.
- `data/processed/mp02_nyala_8k8.wav`, yaitu versi 8 kHz/8-bit.
- `results/figures/mp02_sampling_quantization.png` untuk membandingkan waveform, spektrum, dan ukuran data.
- `results/metrics/mp02_sampling_quantization.json` berisi ukuran file, pengurangan data, dan SNR sederhana.
- `config/audio_format_selected.json` yang menetapkan 16 kHz, mono, 16-bit sebagai format baku proyek.
- `project_status.json` dengan status `ready_for_mp03`.

Mengapa Format 16 kHz/16-bit Dipertahankan?

Eksperimen memang membuat format yang lebih ringan, tetapi proyek serial tidak otomatis memilih file terkecil. Keyword spotting membutuhkan kompromi antara informasi spektral, kualitas amplitudo, dan biaya komputasi. Format 16 kHz/16-bit dipertahankan sebagai baseline karena bandwidth Nyquist 8 kHz sudah mencakup komponen penting ucapan untuk tugas perintah suara, sementara PCM 16-bit menjaga resolusi amplitudo yang nyaman untuk eksperimen berikutnya. Versi 8 kHz dan/atau 8-bit tetap disimpan sebagai bukti trade-off dan dapat digunakan lagi saat membahas optimasi perangkat.

Perangkat dan Perangkat Lunak

Gunakan komputer dan instalasi Python yang sama seperti Mini Project 1. Paket NumPy, SciPy, dan Matplotlib sudah termasuk dalam dependensi Mini Project 1, sehingga tidak diperlukan instalasi baru. Tidak ada proses perekaman pada mini project ini.

Prosedur Langkah demi Langkah

Langkah 1. Periksa Handoff Mini Project 1

Buka terminal di folder `audio_ai_embedded_project` dan pastikan tiga file berikut tersedia. Jangan mengubah nama atau memindahkannya.

```
config/project_config.json
data/raw/nyala_001.wav
project_status.json
```

Buka `project_status.json` secara cepat. Nilai status harus `ready_for_mp02`. Jika tidak, selesaikan kembali validasi Mini Project 1 sebelum melanjutkan.

Langkah 2. Buat Skrip Mini Project 2

Buat file `mini_project_02.py` di folder utama proyek. Skrip berikut membaca audio dan konfigurasi lama, membuat tiga variasi sampling/kuantisasi, mengukur perubahan ukuran data dan SNR sederhana, membuat gambar perbandingan, menyimpan keputusan format, serta memperbarui status proyek.

```
from pathlib import Path
import json
import math
import os
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy.signal import resample_poly

ROOT = Path(__file__).resolve().parent
CONFIG_PATH = ROOT / "config/project_config.json"
STATUS_PATH = ROOT / "project_status.json"

# -----
# 1. VALIDASI HANDOFF MINI PROJECT 1
# -----
with open(CONFIG_PATH, "r", encoding="utf-8") as f:
    config = json.load(f)
with open(STATUS_PATH, "r", encoding="utf-8") as f:
    status = json.load(f)

if status.get("status") != "ready_for_mp02":
    raise RuntimeError("Mini Project 1 belum berstatus ready_for_mp02")

input_wav = ROOT / config["reference_file"]
if not input_wav.exists():
    raise FileNotFoundError(input_wav)

fs, raw = wavfile.read(input_wav)
if raw.ndim > 1:
    raw = raw.mean(axis=1)

# Konversi input PCM menjadi float sekitar [-1, 1]
if raw.dtype == np.uint8:
    x = (raw.astype(np.float32) - 128.0) / 128.0
elif np.issubdtype(raw.dtype, np.integer):
    scale = float(max(abs(np.iinfo(raw.dtype).min), np.iinfo(raw.dtype).max))
    x = raw.astype(np.float32) / scale
else:
    x = raw.astype(np.float32)
x = np.clip(x, -1.0, 1.0)

OUT = ROOT / "data/processed"
FIG = ROOT / "results/figures"
MET = ROOT / "results/metrics"
CFG = ROOT / "config"
for p in [OUT, FIG, MET, CFG]:
    p.mkdir(parents=True, exist_ok=True)

# -----
# 2. RESAMPLING: 16 kHz -> 8 kHz
```

```

# -----
target_fs = 8000
g = math.gcd(int(fs), target_fs)
x_8k = resample_poly(x, target_fs // g, int(fs) // g)
x_8k16 = np.int16(np.clip(x_8k, -1, 1) * 32767)
path_8k16 = OUT / "mp02_nyala_8k16.wav"
wavfile.write(path_8k16, target_fs, x_8k16)

# -----
# 3. KUANTISASI 8-BIT PADA 16 kHz
# WAV 8-bit menggunakan unsigned integer 0..255
# -----
def quantize_u8(signal):
    return np.round((np.clip(signal, -1, 1) + 1.0) * 127.5).astype(np.uint8)

def dequantize_u8(q):
    return q.astype(np.float32) / 127.5 - 1.0

q_16k8 = quantize_u8(x)
path_16k8 = OUT / "mp02_nyala_16k8.wav"
wavfile.write(path_16k8, int(fs), q_16k8)

q_8k8 = quantize_u8(x_8k)
path_8k8 = OUT / "mp02_nyala_8k8.wav"
wavfile.write(path_8k8, target_fs, q_8k8)

# -----
# 4. METRIK SEDERHANA
# -----
def snr_db(reference, test):
    n = min(len(reference), len(test))
    reference = reference[:n]
    test = test[:n]
    noise = reference - test
    ps = np.mean(reference ** 2) + 1e-12
    pn = np.mean(noise ** 2) + 1e-12
    return float(10.0 * np.log10(ps / pn))

x_q8 = dequantize_u8(q_16k8)
quant_snr = snr_db(x, x_q8)

# Resample 8 kHz kembali ke fs asli untuk perbandingan numerik kasar
g2 = math.gcd(target_fs, int(fs))
x_8k_back = resample_poly(x_8k, int(fs) // g2, target_fs // g2)
resample_snr = snr_db(x, x_8k_back)

files = {
    "16k16_baseline": input_wav,
    "8k16": path_8k16,
    "16k8": path_16k8,
    "8k8": path_8k8,
}
sizes = {k: os.path.getsize(v) for k, v in files.items()}
base_size = sizes["16k16_baseline"]
reduction = {
    k: float(100.0 * (1.0 - v / base_size))
    for k, v in sizes.items()
}

metrics = {
    "source_file": str(input_wav.relative_to(ROOT)),
    "source_sample_rate_hz": int(fs),
    "duration_s": float(len(x) / fs),
    "file_size_bytes": sizes,
    "size_reduction_percent_vs_baseline": reduction,
    "quantization_8bit_snr_db": quant_snr,
    "resample_8k_roundtrip_snr_db": resample_snr,
    "selected_format": {
        "sample_rate_hz": 16000,
        "channels": 1,
        "bit_depth": 16,
        "reason": "Baseline proyek: menjaga informasi ucapan dan memudahkan analisis DSP/AI berikutnya"
    }
}
with open(MET / "mp02_sampling_quantization.json", "w", encoding="utf-8") as f:

```

```

    json.dump(metrics, f, indent=2)

selected = {
    "sample_rate_hz": 16000,
    "channels": 1,
    "bit_depth": 16,
    "reference_file": config["reference_file"],
    "selected_by_mini_project": 2
}
with open(CFG / "audio_format_selected.json", "w", encoding="utf-8") as f:
    json.dump(selected, f, indent=2)

# Simpan juga keputusan ke project_config tanpa menghapus konfigurasi lama
config["audio_format_selected"] = selected
with open(CONFIG_PATH, "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

# -----
# 5. VISUALISASI RINGKAS
# -----
fig, ax = plt.subplots(3, 1, figsize=(9, 8))

# Waveform pendek agar langkah kuantisasi terlihat
n_show = min(int(0.04 * fs), len(x))
t = np.arange(n_show) / fs * 1000.0
ax[0].plot(t, x[:n_show], label="16-bit", linewidth=1.0)
ax[0].step(t, x.q8[:n_show], where="mid", label="8-bit", linewidth=0.8)
ax[0].set_title("Efek Kuantisasi pada Potongan Waveform")
ax[0].set_xlabel("Waktu (ms)")
ax[0].set_ylabel("Amplitudo")
ax[0].legend()
ax[0].grid(alpha=0.25)

# Spektrum baseline dan hasil 8 kHz
def spectrum(sig, sr):
    w = np.hanning(len(sig))
    X = np.abs(np.fft.rfft(sig * w))
    f = np.fft.rfftfreq(len(sig), 1.0 / sr)
    Xdb = 20 * np.log10(X / (np.max(X) + 1e-12) + 1e-12)
    return f, Xdb

f16, s16 = spectrum(x, fs)
f8, s8 = spectrum(x.q8, target_fs)
ax[1].plot(f16, s16, label="16 kHz")
ax[1].plot(f8, s8, label="8 kHz")
ax[1].set_xlim(0, fs / 2)
ax[1].set_ylim(-80, 5)
ax[1].set_title("Spektrum: Batas Nyquist Berubah saat Downsampling")
ax[1].set_xlabel("Frekuensi (Hz)")
ax[1].set_ylabel("Magnitudo relatif (dB)")
ax[1].legend()
ax[1].grid(alpha=0.25)

labels = ["16k/16", "8k/16", "16k/8", "8k/8"]
values = [sizes[k] / 1024 for k in ["16k16_baseline", "8k16", "16k8", "8k8"]]
ax[2].bar(labels, values)
ax[2].set_title("Perbandingan Ukuran File")
ax[2].set_ylabel("Ukuran (KiB)")
ax[2].grid(axis="y", alpha=0.25)

fig.tight_layout()
fig.savefig(FIG / "mp02_sampling_quantization.png", dpi=160)
plt.close(fig)

# -----
# 6. HANDOFF KE MINI PROJECT 3
# -----
status = {
    "last_completed_mini_project": 2,
    "next_mini_project": 3,
    "next_input": config["reference_file"],
    "audio_format": "config/audio_format_selected.json",
    "metrics": "results/metrics/mp02_sampling_quantization.json",
    "status": "ready_for_mp03"
}

```

```

with open(STATUS_PATH, "w", encoding="utf-8") as f:
    json.dump(status, f, indent=2)

print("Mini Project 2 selesai")
print("SNR kuantisasi 8-bit : ", round(quant_snr, 2), "dB")
print("SNR resampling 8 kHz : ", round(resample_snr, 2), "dB")
print("Format dipilih : 16 kHz / mono / 16-bit")
print("Status berikutnya : ready_for_mp03")

```

Langkah 3. Jalankan Eksperimen

Jalankan skrip dari folder utama proyek. Tidak ada input manual dan tidak ada perekaman baru.

```
python mini_project_02.py
```

Jika skrip berhenti dengan pesan bahwa Mini Project 1 belum ready_for_mp02, periksa project_status.json. Jika input WAV tidak ditemukan, jangan membuat file pengganti; gunakan kembali artefak Mini Project 1 sesuai nama yang ditetapkan.

Langkah 4. Dengarkan dan Bandingkan Empat File

Putar nyala_001.wav, mp02_nyala_8k16.wav, mp02_nyala_16k8.wav, dan mp02_nyala_8k8.wav. Catat secara singkat apakah perubahan terdengar jelas. Untuk ucapan satu kata, penurunan 16 kHz ke 8 kHz mungkin masih mempertahankan keterpahaman, tetapi bandwidth yang tersedia menjadi lebih sempit. Kuantisasi 8-bit biasanya menambah kekasaran/noise kuantisasi dibanding PCM 16-bit.

Langkah 5. Baca Gambar dan Metrik

Buka results/figures/mp02_sampling_quantization.png dan results/metrics/mp02_sampling_quantization.json. Jawab tiga pertanyaan berikut pada catatan praktikum:

- Konfigurasi mana yang menghasilkan ukuran file paling kecil?
- Apa yang terjadi pada batas frekuensi ketika sampling rate turun dari 16 kHz menjadi 8 kHz?
- Mengapa proyek tetap memilih 16 kHz/16-bit walaupun file 8 kHz/8-bit lebih kecil?

Langkah 6. Validasi Handoff ke Mini Project 3

Artefak	Yang diperiksa	Kriteria lulus
Input MP1	data/raw/nyala_001.wav tetap ada	Tidak ada perekaman ulang
Varian sampling	mp02_nyala_8k16.wav dapat diputar	Resampling berhasil
Varian kuantisasi	mp02_nyala_16k8.wav dan mp02_nyala_8k8.wav dapat diputar	Kuantisasi berhasil
Gambar	mp02_sampling_quantization.png terbuka	Perbandingan visual tersedia
Metrik	mp02_sampling_quantization.json berisi size dan SNR	Eksperimen terdokumentasi
Format terpilih	audio_format_selected.json = 16 kHz, mono, 16-bit	Parameter baku untuk MP3
Status proyek	status = ready_for_mp03	Siap melanjutkan Bab 3

Interpretasi Singkat

Mini Project 2 menunjukkan bahwa sampling rate dan bit depth merupakan keputusan sistem, bukan sekadar parameter format file. Menurunkan sampling rate mengurangi jumlah sampel per detik dan sekaligus menurunkan frekuensi Nyquist. Menurunkan bit depth mengurangi jumlah bit per sampel, tetapi meningkatkan galat kuantisasi. Kedua tindakan menghemat penyimpanan dan beban pemrosesan, namun dapat membuang informasi yang kelak dibutuhkan oleh fitur dan model.

Untuk running project keyword spotting, 16 kHz/16-bit dipilih sebagai format kerja sampai ada alasan eksperimental untuk mengubahnya. Pilihan ini sengaja disimpan dalam config/audio_format_selected.json agar Bab 3 dan bab-bab berikutnya tidak menggunakan parameter yang berbeda secara tidak sengaja.

Artefak yang Wajib Dibawa ke Mini Project 3

Mini Project 3 harus menggunakan folder proyek yang sama dan membaca artefak berikut:

- data/raw/nyala_001.wav sebagai sinyal referensi utama.
- config/audio_format_selected.json sebagai sumber sampling rate dan format baku.
- results/metrics/mp02_sampling_quantization.json sebagai catatan trade-off sampling/kuantisasi.

Mini Project 3 tidak melakukan perekaman baru. Mahasiswa memakai audio referensi terpilih untuk menghitung FFT/STFT dan membuat representasi waktu-frekuensi. Jika project_status.json sudah berisi ready_for_mp03, Mini Project 2 dinyatakan selesai.

Catatan: total aktivitas praktis ini dirancang setelah lingkungan Python dari Mini Project 1 tersedia. Jika waktu terbatas, proses mendengarkan empat file dapat dibatasi pada satu kali pemutaran per file tanpa mengurangi handoff ke Mini Project 3.

Referensi Bab 2

1. Oppenheim, A. V., & Schaffer, R. W. (2010). Discrete-Time Signal Processing (3rd ed.). Pearson.
2. Lyons, R. G. (2011). Understanding Digital Signal Processing (3rd ed.). Pearson.
3. Zölzer, U. (Ed.). (2008). Digital Audio Signal Processing (2nd ed.). Wiley.
4. Rabiner, L. R., & Schaffer, R. W. (2010). Theory and Applications of Digital Speech Processing. Pearson.
5. Pohlmann, K. C. (2010). Principles of Digital Audio (6th ed.). McGraw-Hill.
6. Watkinson, J. (2001). The Art of Digital Audio (3rd ed.). Focal Press.
7. Warden, P., & Situnayake, D. (2019). TinyML. O'Reilly Media.

Akhir Bab 2